React

1.1 THE REAL WORLD PROBLEM (THE ROOT OF EVERYTHING)

Imagine you are working on a **real e-commerce website**.

This website has:

- A home page
- A products page
- A favorites page

Each page shows **products**.

Each product has:

- An image
- A title
- A price
- A button

Visually, every product looks **exactly the same**, only the data changes.

1.2 The First Version (Pure HTML Thinking)

When beginners build this website, they usually do this:

They copy the same HTML block again and again.

```HTML
<div class="product">
  <img src="shoe.jpg" />
  <h3>Nike Shoe</h3>
  <p>$120</p>
  <button>Add to Cart</button>
</div>
```

They paste it:

- On the home page
- On the products page
- On the favorites page

At first, this looks fine.

1.3 Why This Becomes a Serious Problem

Question 1

What if the designer says:

> “We want to change the button text to ‘Buy Now’”

You must:

- Find every product card
- Change every button manually

Question 2

What if the designer says:

> “Add a discount badge to products on sale”

Now you must:

- Edit every product card
- Make sure none are missed

Question 3

What if your teammate forgets to update one page?

Now:

- The UI is inconsistent
- Bugs appear
- The app looks unprofessional

This problem is **not small**.
This is one of the **biggest problems in frontend development**.

1.4 Naming the Problem

This problem is called:

Code Duplication

Code duplication means:

- The same code exists in many places
- Changes are expensive
- Maintenance is hard
- Teams suffer

2.1 How Humans Naturally Think

When humans see repetition, they say:

> “This should exist once, not many times.”

This is not React thinking yet.
This is **software engineering thinking**.

2.2 Reusable Pieces

Instead of thinking in pages, we start thinking in **pieces**:

- A product card
- A navbar
- A footer

Each piece:

- Has one responsibility
- Can be reused
- Can be tested alone

This idea leads us to the concept of **components**.

WHAT IS A COMPONENT (WITHOUT REACT YET)

3.1 Component – Plain Explanation

A **component** is:

- A reusable piece of UI
- That does one thing
- And does it well

A component:

- Receives data
- Displays UI
- Does not care where it is used

3.2 Mental Analogy

Think of a component like:

- A function
- A machine
- A factory

Input → Processing → Output

For UI:

- Input → data
- Output → UI

3.3 Product Card as a Component (Conceptually)

Instead of copying HTML, we want something like:

> “Here is my product card structure.
> Now just give it different data.”

This idea **cannot be solved cleanly with HTML alone**.

TRYING TO SOLVE IT WITH VANILLA JAVASCRIPT

4.1 Dynamic UI with JavaScript

Let’s try to generate products using JavaScript.

```html
<div id="products"></div>

<script>
  const products = [
    { name: "Nike Shoe", price: 120 },
    { name: "Adidas Shoe", price: 100 },
    { name: "Puma Shoe", price: 90 }
  ];

  const container = document.getElementById("products");

  products.forEach(product => {
    const card = document.createElement("div");
    card.innerHTML = `
      <h3>${product.name}</h3>
      <p>$${product.price}</p>
      <button>Add to Cart</button>
    `;
    container.appendChild(card);
  });
</script>
```

This works.
So what is the problem?

4.2 Why This Approach Breaks at Scale

Let's analyze carefully.

Problem 1: Manual DOM Manipulation

- You manually create elements
- You manually update text
- You manually append nodes

Problem 2: Logic and UI Are Mixed

- Data logic
- Rendering logic
- Event handling
  All in one place

Problem 3: State Changes Are Painful

Imagine:

- Clicking "Add to Cart"
- Updating cart count
- Updating UI in multiple places

You must:

- Track state manually
- Update DOM manually
- Avoid bugs manually

This becomes **very hard very fast**.

REACT’S CORE IDEA

5.1 React’s Philosophy

React says:

> “Stop telling the browser HOW to update the UI.
> Just tell React WHAT the UI should look like.”

This is called:
Declarative Programming

5.2 UI as a Function of Data

In React, we think like this:

```
None
UI = function(data)
```

- Change the data
- React changes the UI automatically

You never say:

- “Create element”
- “Update innerText”
- “Append child”

React does this for you.

WHAT IS REACT

React is:

- A JavaScript library
- For building user interfaces
- Using components
- With automatic UI updates

React focuses ONLY on:

- The UI layer

React is NOT:

- A backend
- A database
- A replacement for JavaScript

WHY REACT IS FAST – VIRTUAL DOM

7.1 The Real DOM Problem

The browser DOM is:

- Heavy
- Slow to update
- Expensive to re-render

Updating the DOM many times causes:

- Performance issues
- Laggy UI

7.2 What React Does Differently

React creates:

- A **Virtual DOM**
- A lightweight JavaScript representation of the UI

7.3 Step-by-Step Virtual DOM Process

1. React builds a Virtual DOM
2. Data changes
3. React builds a new Virtual DOM
4. React compares old vs new (diffing)
5. React updates only what changed

This makes React:

- Fast
- Efficient
- Smart

SINGLE PAGE APPLICATION (SPA)

8.1 Traditional Websites (MPA)

- Each page = new HTML file
- Full reload on navigation
- Slower experience

8.2 React Applications (SPA)

- One HTML file
- JavaScript controls navigation
- No full page reload
- Smooth experience

This is called:
Single Page Application

WHY WE NEED TOOLS (NODE.JS)

9.1 Why Browsers Are Not Enough

Browsers:

- Don’t understand modern JS fully
- Don’t bundle files
- Don’t optimize code

So we need tools.

9.2 Node.js

Node.js allows:

- Running JavaScript outside the browser
- Installing packages
- Running development servers

Important:
Node.js is a **tool**, not React itself.

VITE – REACT DEVELOPMENT TOOL

Vite is:

- A build tool
- A dev server
- A project generator

Vite makes React:

- Fast to start
- Easy to configure
- Modern

INSTALLING REACT WITH VITE (DETAILED)

https://vite.dev/

```shell
npm create vite@latest
```

This:

- Creates project structure
- Asks for framework
- Sets up configuration

```shell
cd name of project
npm install
```

This:

- Downloads dependencies

```shell
npm run dev
```

This:

- Starts the development server

Understanding Your Vite + React Project

When you create a React project using **Vite**, it generates a **ready-to-work development environment**.
Each file has a **job**.
Nothing here is random.

We will go **from outside** → **inside**, just like a computer does.

Big Picture First (Very Important)

Your React app is made of **three layers**:

1. **Configuration & tools** (Vite, ESLint, npm)
2. **HTML entry point** (the single page)
3. **React code** (components, JSX, CSS)

Keep this mental model in your head while reading.

`node_modules/`

What is this folder?

This folder contains:

- All **external libraries**
- React itself
- Vite internals
- Any package installed via `npm`

Think of it like:

A **storage room** full of tools you didn't build yourself.

Important rules:

- NEVER edit this folder
- NEVER upload it to GitHub
- If something breaks → delete it and run `npm install`

This folder is created automatically.

`public/`

What is this folder?

This folder contains **static files**.

Example:

- Images
- Icons
- SVGs
- Files that do NOT change

In your project:

```None
- public/
- └── vite.svg
```

Important behavior:

- Files here are **not processed by React**
- They are copied **as-is** to the final build

When should students use `public/`?

- Logos
- Static images
- Files that don’t depend on JS logic

`src/`

This is where **YOU work**.
Almost all React code lives here.

Let's go file by file.

`src/assets/`

What is this folder?

This is for:

- Images
- Fonts
- SVGs
- Assets that **ARE used inside React code**

Difference from `public/`:

- Assets here are **imported**
- They are processed by Vite

Example:

```javascript
import logo from "./assets/logo.png";
```

- **Used in JSX?** → `src/assets`
- **Just static?** → `public`

`src/App.jsx`

What is `App.jsx`?

This is the **root React component**.

Every React app has:

- One main component
- Everything else lives inside it

Example:

```
None

function App() {
  return <h1>Hello React</h1>;
}

export default App;
```

Important ideas:

- `App` is a **component**
- It returns JSX
- Other components will be placed inside it later

Think of `App` as:

> “The main container of the UI”

`src/App.css`

What is this file?

This file contains:

- CSS styles **specific to `App.jsx`**

Why is it separated?

- Organization
- Cleaner code
- Easier maintenance

Example:

```CSS
- h1 {
-   color: blue;
- }
```

- CSS can be global or component-based
- This is a **component-level style file**

`src/index.css`

What is `index.css`?

This is the **global CSS file**.

It usually contains:

- Reset styles
- Body styles
- Fonts
- Global rules

Example:

```
CSS
- body {
-   margin: 0;
-   font-family: sans-serif;
- }
```

Difference from `App.css`:

`index.css`	`App.css`
Global	Component-related
Affects whole app	Affects App
Loaded once	Used where imported

`src/main.jsx`

What is `main.jsx`?

This file is the **bridge** between:

- The browser
- React

This is where React is **attached to the HTML page**.

Typical code:

None

```
- import React from "react";
- import ReactDOM from "react-dom/client";
- import App from "./App";
- import "./index.css";
-
- ReactDOM.createRoot(document.getElementById("root")).rend
  er(
-   <App />
- );
```

What is happening step-by-step?

1. The browser loads `index.html`
2. Vite loads `main.jsx`
3. React finds `<div id="root">`
4. React renders `<App />` inside it

This file usually **never changes much**

`index.html`

Why is this file outside `src`?

Because this is the **actual HTML page** the browser loads.

Typical content:

```HTML
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
```

Key idea:

- React does NOT replace HTML
- React **lives inside** this div

This is why React apps are called:
Single Page Applications

There is:

- One HTML file
- One root div
- Everything else is React

`.gitignore`

What is this file?

It tells Git:

> "Do NOT track these files"

Usually ignores:

- `node_modules`
- `.env`
- build files

Very important for teamwork and GitHub.

`package.json`

What is `package.json`?

This is the **identity card** of your project.

It contains:

- Project name
- Dependencies
- Scripts
- Project type

Example:

```JSON
{
  "scripts": {
    "dev": "vite",
    "build": "vite build"
  }
}
```

- `npm run dev` comes from here
- Libraries are listed here
- This file defines how the project runs

`package-lock.json`

What is this file?

This file:

- Locks exact dependency versions
- Ensures everyone uses the same versions

`eslint.config.js`

What is ESLint?

ESLint is a **code quality tool**.

It:

- Warns about mistakes
- Enforces best practices
- Helps write cleaner code

Example warnings:

- Unused variables
- Bad syntax
- Dangerous patterns

ESLint does NOT run your app
It helps you write **better code**

vite.config.js

What is this file?

This is the **configuration file for Vite**.

It controls:

- Plugins
- Aliases
- Build behavior

Example:

```javascript
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
});
```

- You usually don't touch this at first
- Advanced customization happens here

README.md

What is this file?

This is:

- Documentation
- Instructions
- Project explanation

Used for:

- GitHub
- Onboarding
- Explaining how to run the project

JSX in React

Before JSX: How Does React Describe UI?

Before JSX, let's ask a very basic question:

> **How does React know what to show on the screen?**

The browser understands:

- HTML
- CSS
- JavaScript

React is written in **JavaScript**, not HTML.

So React must describe the UI **using JavaScript**.

Life Without JSX

JSX is **NOT required** to use React.
It just makes life easier.

Here is React **without JSX**:

```javascript
const element = React.createElement(
  "h1",
  null,
  "Hello React"
);
```

This creates:

```html
<h1>Hello React</h1>
```

What is happening?

- `"h1"` → element type
- `null` → props
- `"Hello React"` → content

This is:

- Hard to read
- Hard to write
- Hard to maintain

Now imagine a full page written like this

Why JSX Was Created

JSX was created to solve **one problem**:

> Writing UI in pure JavaScript is painful and unreadable.

JSX allows us to write:

```None
<h1>Hello React</h1>
```

Instead of:

```JavaScript
React.createElement("h1", null, "Hello React");
```

What JSX REALLY Is

Big Truth:

JSX is NOT HTML

JSX is:

- A **syntax extension** for JavaScript
- Converted to JavaScript before running
- Just syntax sugar

When your code runs:
JSX disappears

JSX Compilation (Behind the Scenes)

This JSX:

```
None
<h1>Hello</h1>
```

Is transformed into:

```
JavaScript
React.createElement("h1", null, "Hello");
```

This happens:

- Before the browser sees the code
- By Vite / Babel

📌 The browser NEVER sees JSX.

Why JSX Looks Like HTML (But Isn't)

JSX was designed to:

- Look familiar
- Be readable
- Feel like writing UI

But JSX follows **JavaScript rules**, not HTML rules.

This is why JSX behaves differently.

JSX Rules

JSX Must Return ONE Parent Element

This is invalid:

```
None
return (
  <h1>Hello</h1>
  <p>Welcome</p>
);
```

This is valid:

```
None
return (
  <div>
    <h1>Hello</h1>
    <p>Welcome</p>
  </div>
);
```

Why?

Because `React.createElement` can return **only one object**.

Use `className`, NOT `class`

JSX is JavaScript, and `class` is a reserved word.

Wrong:

```
None
<div class="box"></div>
```

Correct:

```
None
<div className="box"></div>
```

Attributes Use camelCase

HTML:

```
HTML
onclick="doSomething()"
```

JSX:

```
None
onClick={doSomething}
```

JavaScript Inside JSX `{}` (CORE CONCEPT)

JSX allows JavaScript **expressions** inside `{}`.

Example 1: Variables

```
None
const name = "Dania";

<h1>Hello {name}</h1>
```

Result:

```
None
Hello Dania
```

Example 2: Expressions

```
None
const price = 100;

<p>Total: {price * 2}</p>
```

JSX does NOT allow:

- `if`
- `for`
- `while`

But it allows:

- Expressions
- Ternaries
- Logical operators

Conditional Rendering in JSX

This will NOT work:

```
None
{if (isLoggedIn) {
  <p>Welcome</p>
}}
```

Correct Way 1: Logical AND

```
None
{isLoggedIn && <p>Welcome</p>}
```

Correct Way 2: Ternary Operator

```
None
{isLoggedIn ? <p>Welcome</p> : <p>Please log in</p>}
```

Rendering Lists in JSX (`map`)

JSX does NOT support loops directly.

This will NOT work:

```
None
for (let i = 0; i < items.length; i++) {
  <p>{items[i]}</p>
}
```

Correct Way: `map`

```
None
items.map(item => <p>{item}</p>)
```

Why?

- `map` returns a new array
- JSX can render arrays

JSX + Components

```
None
function ProductCard() {
  return (
    <div>
      <h3>Nike Shoe</h3>
      <p>$120</p>
    </div>
  );
}
```

JSX makes components:

- Readable
- Declarative
- UI-focused

1 JSX Is NOT a Template Language

Important misconception:

JSX is NOT like HTML templates
JSX is NOT string-based

JSX is:

- JavaScript
- Expressions
- Functions
- Logic

Common Beginner Mistakes (Very Important)

Mistake 1: Forgetting `{}`

```
None
<p>price</p>   // shows "price"
<p>{price}</p> // shows value
```

Mistake 2: Writing Logic Instead of Expressions

```
None
{if (x > 5) ...} ❌
{x > 5 && ...}   ✅
```

Mistake 3: Returning Multiple Elements

Always wrap JSX.

React Props – Deep & Clear Explanation

How Components Communicate Using Data

Before We Start: Why Props Exist at All

Before learning *what props are*, we must understand **why they exist**.

React is built on **components**.
Components are **independent**, **reusable**, and **isolated**.

Now ask yourself:

How can one component send information to another component?

That question leads us directly to **props**.

THE REAL PROBLEM PROPS SOLVE

1.1 A Realistic Scenario

Imagine an online shop again.

We have:

- A list of products
- Each product looks the same
- Only the **data changes**

Without props, every component would be **hard-coded**.

Example (BAD):

```
None
function ProductCard() {

  return (

    <div>

      <h3>Nike Shoe</h3>

      <p>$120</p>

    </div>

  );

}
```

This component is:

- Not reusable
- Fixed data
- Useless for real apps

What We ACTUALLY Want

We want to say:

> “Here is the structure of a product card.
> Now I will give it different data.”

This is exactly what **props** do.

WHAT ARE PROPS (CONCEPT FIRST)

Plain Definition

Props (short for *properties*) are:

- Data passed **from a parent component**
- To a **child component**
- As an **object**
- Read-only (cannot be changed by the child)

Props are how components talk to each other.

Props Mental Model

Think of a component like a **function**.

```javascript
function add(a, b) {

  return a + b;

}
```

- `a` and `b` are inputs
- The function uses them
- The function does NOT change them

Now look at a component:

```
<ProductCard name="Nike" price={120} />
```

- `name` and `price` are inputs
- The component uses them
- The component does NOT change them

Props = function parameters for components

FIRST PROPS EXAMPLE

Parent Component

```
None
function App() {
  return (
    <ProductCard name="Nike Shoe" price={120} />
  );
}
```

Here:

- `App` is the **parent**
- `ProductCard` is the **child**

Child Component

```
None
function ProductCard(props) {
  return (
    <div>
      <h3>{props.name}</h3>
      <p>${props.price}</p>
    </div>
  );
}
```

What Is `props` Exactly?

`props` is an **object**.

Inside the component, React creates:

```
JavaScript
props = {

  name: "Nike Shoe",

  price: 120

}
```

So:

- `props.name` → `"Nike Shoe"`
- `props.price` → `120`

DESTRUCTURING PROPS

Why Destructuring Exists

Writing `props.name`, `props.price` again and again is:

- Verbose
- Ugly
- Hard to read

Destructuring Syntax

```
None
function ProductCard({ name, price }) {

  return (

    <div>

      <h3>{name}</h3>

      <p>${price}</p>

    </div>

  );

}
```

This means:

```
JavaScript
const name = props.name;

const price = props.price;
```

This is the **most common way** props are used.

PROPS ARE READ-ONLY (CRITICAL RULE)

This Is NOT Allowed

```
None

function ProductCard({ price }) {

  price = price + 10; //  WRONG

}
```

Why?

Props come from the **parent**.
The child **must not change them**.

This rule exists to:

- Prevent bugs
- Keep data flow predictable
- Keep React apps stable

React has one-way data flow:

Parent → Child

PASSING DIFFERENT TYPES OF PROPS

Strings

```
None
<ProductCard name="Nike Shoe" />
```

Numbers

```
None
<ProductCard price={120} />
```

Curly braces are required for non-strings.

Booleans

```
None
<ProductCard isOnSale={true} />
```

Or shorthand:

```
None
<ProductCard isOnSale />
```

Arrays

```
None
<ProductCard tags={["sport", "new"]} />
```

Objects

```
None

<ProductCard product={{ name: "Nike", price: 120 }} />
```

Functions

```
None

<ProductCard onAddToCart={handleAdd} />
```

This allows **child** → **parent communication** (covered later).

REALISTIC EXAMPLE – PRODUCT LIST

The Data (Parent)

```
None

const products = [

  { id: 1, name: "Nike Shoe", price: 120 },

  { id: 2, name: "Adidas Shoe", price: 100 },

  { id: 3, name: "Puma Shoe", price: 90 }

];
```

Rendering with `map`

```
None
function App() {
  return (
    <div>
      {products.map(product => (
        <ProductCard
          key={product.id}
          name={product.name}
          price={product.price}
        />
      ))}
    </div>
  );
}
```

Child Component

```
None
function ProductCard({ name, price }) {

  return (

    <div>

      <h3>{name}</h3>

      <p>${price}</p>

    </div>

  );

}
```

Important idea:

> One component definition → unlimited UI

WHY `key` EXISTS

When using `map`, React needs to:

- Track elements
- Update efficiently

```
None
<ProductCard key={product.id} />
```

- `key` helps React identify components
- `key` is NOT passed as a prop

`key` is for React, not for you.

DEFAULT PROPS

```
None
function ProductCard({ name = "Unknown", price = 0 }) {
  return (
    <div>
      <h3>{name}</h3>
      <p>${price}</p>
    </div>
  );
}
```

This prevents:

- `undefined`
- Broken UI

COMMON BEGINNER MISTAKES

Mistake 1: Forgetting `{}`

```
None
<ProductCard price="120" /> // string
<ProductCard price={120} /> // number
```

Mistake 2: Trying to Change Props

```
None
props.price = 200; //  NEVER
```

Mistake 3: Using Props Without Passing Them

```
None
<h3>{name}</h3> // ❌ name not defined
```

PROPS VS STATE (PREVIEW)

Props	State
Passed from parent	Internal
Read-only	Can change
Used for data	Used for logic

We will study **state** next.